1495 UNIVERSITY OF ABERDEEN

JC4004 – Computational Intelligence

Sessions 19 & 20: Convolutional neural networks

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)
Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)
Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

Oct-Dec 2025

Why convolutional neural networks?

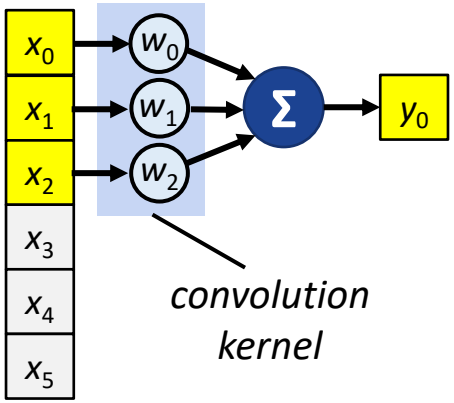
- The baseline perceptron architecture is not optimal for processing some types of input data, especially images
  - A linear layer taking a 100×100 pixel image as input would require $10{,}000$ weights for *one neuron only*
- *Convolutional neural network (CNN)* solves many of the problems of perceptrons
  - Convolution operation processes only part of the input (e.g., a tile of an image) at once using a smaller number of weights per layer
  - Convolutional neural networks also introduced other new layers for e.g., pooling and normalisation

Convolution operation

- In the context of machine learning, we can define basic 1D convolution as:

$$y_i = b + \sum_{j=0}^{K-1} w_j x_{i+j}$$

  - Here, x is the input vector, y is the output vector, and w is the convolution kernel of size K, bias b not always used
  - Note that different definition of convolution is used in signal processing!

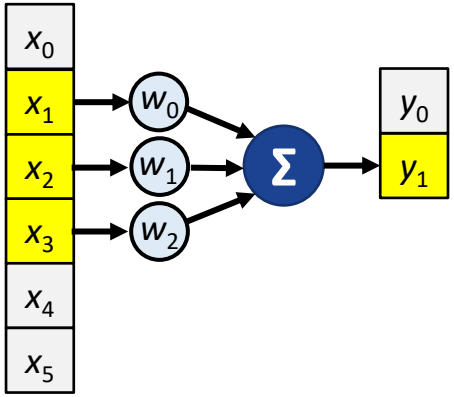


Convolution operation

- In the context of machine learning, we can define basic 1D convolution as:

$$y_i = b + \sum_{j=0}^{K-1} w_j x_{i+j}$$

  - Here, x is the input vector, y is the output vector, and w is the convolution kernel of size K, bias b not always used
  - Note that different definition of convolution is used in signal processing!

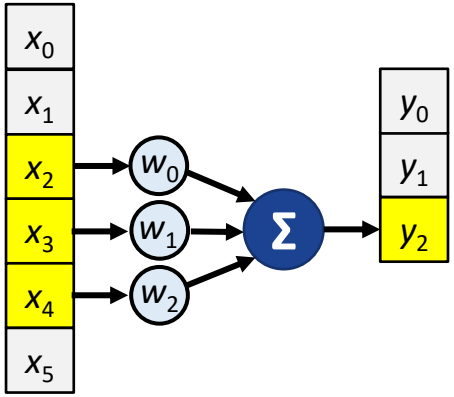


Convolution operation

- In the context of machine learning, we can define basic 1D convolution as:

$$y_i = b + \sum_{j=0}^{K-1} w_j x_{i+j}$$

  - Here, x is the input vector, y is the output vector, and w is the convolution kernel of size K, bias b not always used
  - Note that different definition of convolution is used in signal processing!

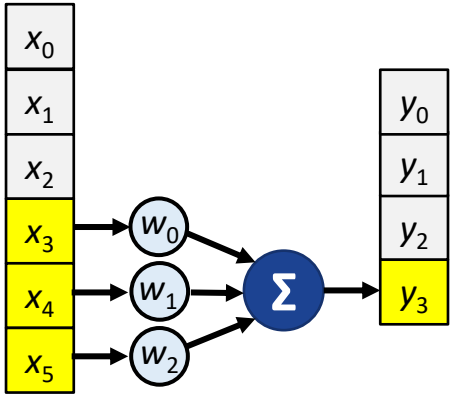


Convolution operation

- In the context of machine learning, we can define basic 1D convolution as:

$$y_i = b + \sum_{j=0}^{K-1} w_j x_{i+j}$$

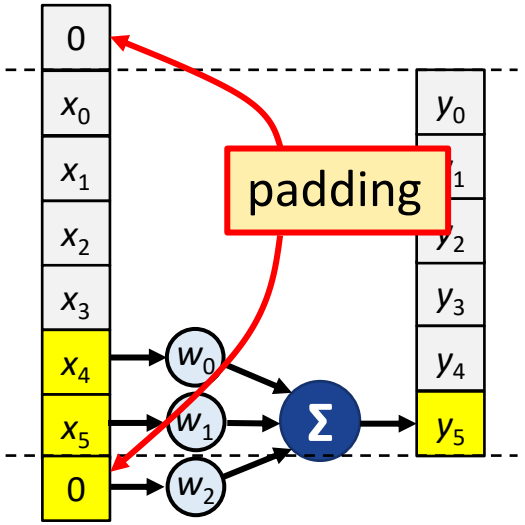
  - Here, x is the input vector, y is the output vector, and w is the convolution kernel of size K, bias b not always used
  - Note that different definition of convolution is used in signal processing!



Padding and stride

- For kernels with length $K > 1$, output vector is shorter than the input vector
  - Optional padding with dummy values (usually zeros) can be added to the input to make the input and output equal in size
- The output vector size can be reduced by using *stride* (step size) larger than 1
  - Given input size L_{IN}, padding P, and stride S, the output size L_{OUT} can be computed as:

$$L_{OUT} = \frac{L_{IN} - K + 2P}{S} + 1$$

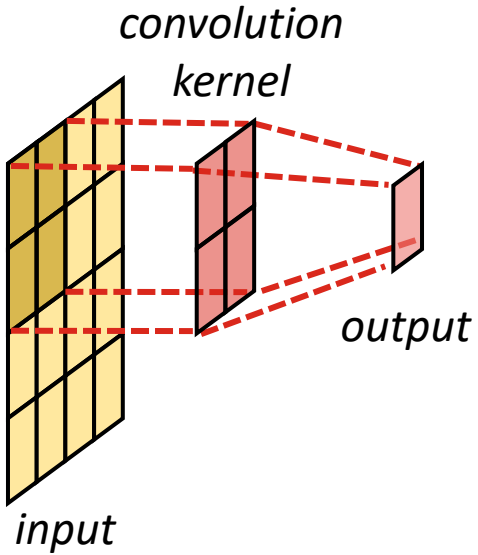


2D convolution

- Convolution can be defined also for 2D input (e.g., monochrome images):

$$y_{i,j} = b + \sum_{m=0}^{K_H-1} \sum_{n=0}^{K_W-1} w_{m,n} x_{i+m,j+n}$$

  - Input x, output y, and convolution kernel w are 2D matrices, and the kernel height is K_H and the width is K_W
  - Usually $K_H = K_W$, and padding and stride are the same for both horizontal and vertical directions

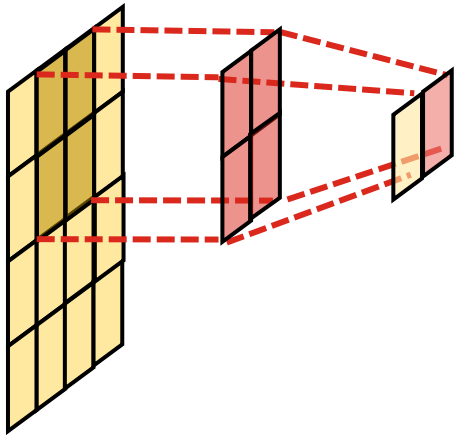


2D convolution

- Convolution can be defined also for 2D input (e.g., monochrome images):

$$y_{i,j} = b + \sum_{m=0}^{K_H-1} \sum_{n=0}^{K_W-1} w_{m,n} x_{i+m,j+n}$$

  - Input x, output y, and convolution kernel w are 2D matrices, and the kernel height is K_H and the width is K_W
  - Usually $K_H = K_W$, and padding and stride are the same for both horizontal and vertical directions

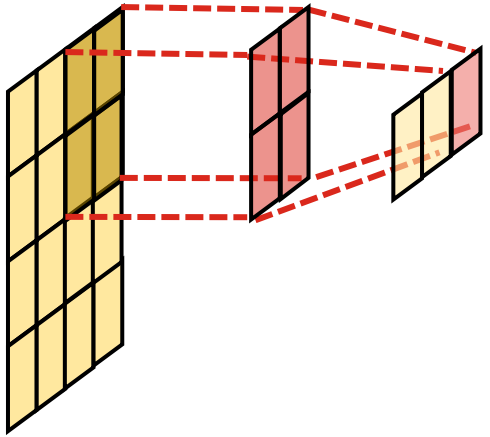


2D convolution

- Convolution can be defined also for 2D input (e.g., monochrome images):

$$y_{i,j} = b + \sum_{m=0}^{K_H-1} \sum_{n=0}^{K_W-1} w_{m,n} x_{i+m,j+n}$$

  - Input x, output y, and convolution kernel w are 2D matrices, and the kernel height is K_H and the width is K_W
  - Usually $K_H = K_W$, and padding and stride are the same for both horizontal and vertical directions

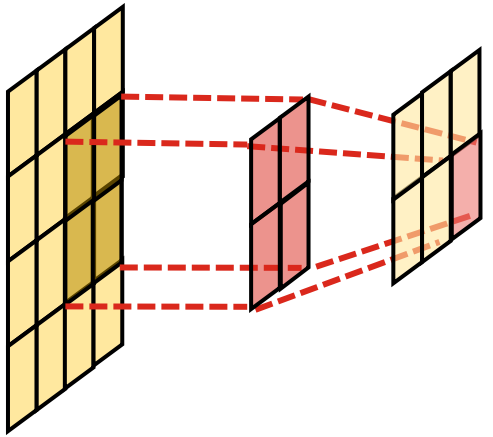


2D convolution

- Convolution can be defined also for 2D input (e.g., monochrome images):

$$y_{i,j} = b + \sum_{m=0}^{K_H-1} \sum_{n=0}^{K_W-1} w_{m,n} x_{i+m,j+n}$$

  - Input x, output y, and convolution kernel w are 2D matrices, and the kernel height is K_H and the width is K_W
  - Usually $K_H = K_W$, and padding and stride are the same for both horizontal and vertical directions

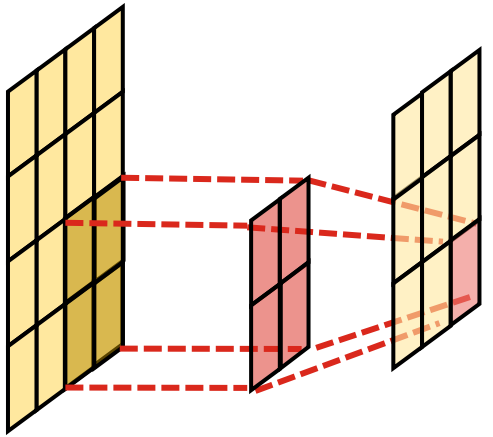


2D convolution

- Convolution can be defined also for 2D input (e.g., monochrome images):

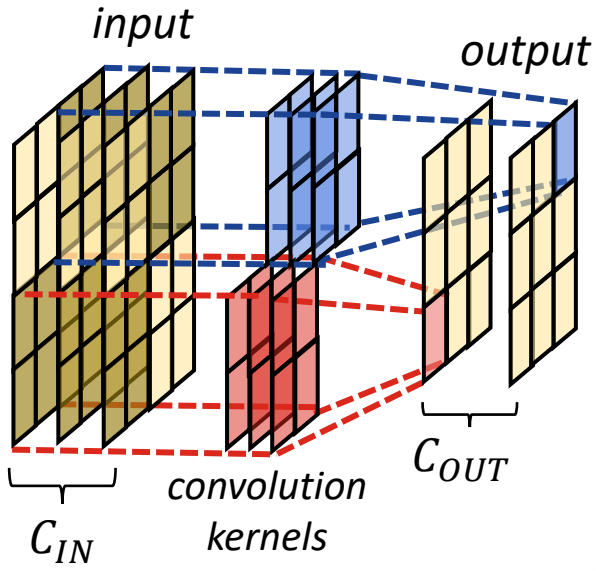
$$y_{i,j} = b + \sum_{m=0}^{K_H-1} \sum_{n=0}^{K_W-1} w_{m,n} x_{i+m,j+n}$$

  - Input x, output y, and convolution kernel w are 2D matrices, and the kernel height is K_H and the width is K_W
  - Usually $K_H = K_W$, and padding and stride are the same for both horizontal and vertical directions



2D convolution for multiple channels

- In practice, 2D data often has multiple channels
  - For example, colour images have R, G, and B channels, so the input size is $H \times W \times 3$
  - 2D convolutions usually cover all the input channels; so, the kernel size for $K \times K$ convolution is actually $K \times K \times C_{IN}$, where C_{IN} is the number of input channels
  - Each output channel has its own convolution kernel, so for C_{OUT} output channels, there are C_{OUT} kernels



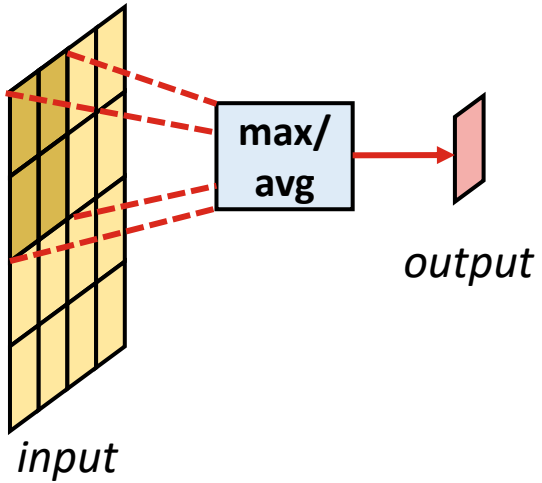
Max and average pooling

- *Max* and *average pooling* (*MaxPool* and *AvgPool*) are typically used for reducing spatial resolution (usually, stride $S > 1$)
  - MaxPool and AvgPool use sliding window like convolution, but convolution operation replaced by max or averaging operation

    MaxPool: $y_{i,j} = \max(\{x_{Si,Sj}, \ldots, x_{Si+K-1,Sj+K-1}\})$

    AvgPool: $y_{i,j} = \text{mean}(\{x_{Si,Sj}, \ldots, x_{Si+K-1,Sj+K-1}\})$

  - For multichannel input, performed separately for each input channel: $C_{OUT} = C_{IN}$



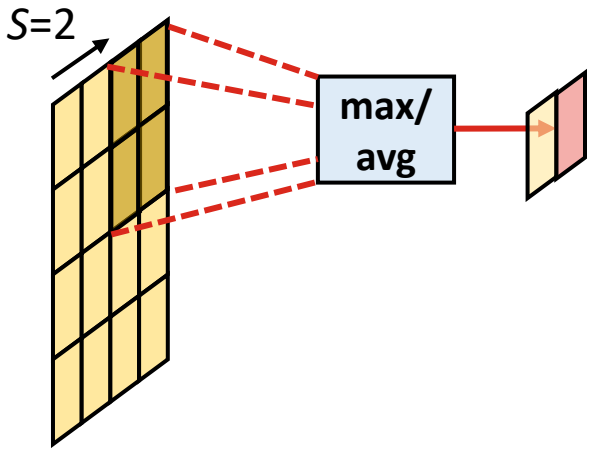
Max and average pooling

- *Max* and *average pooling* (*MaxPool* and *AvgPool*) are typically used for reducing spatial resolution (usually, stride $S > 1$)
  - MaxPool and AvgPool use sliding window like convolution, but convolution operation replaced by max or averaging operation

    MaxPool: $y_{i,j} = \max(\{x_{Si,Sj}, \dots, x_{Si+K-1,Sj+K-1}\})$

    AvgPool: $y_{i,j} = \text{mean}(\{x_{Si,Sj}, \dots, x_{Si+K-1,Sj+K-1}\})$

  - For multichannel input, performed separately for each input channel: $C_{OUT} = C_{IN}$



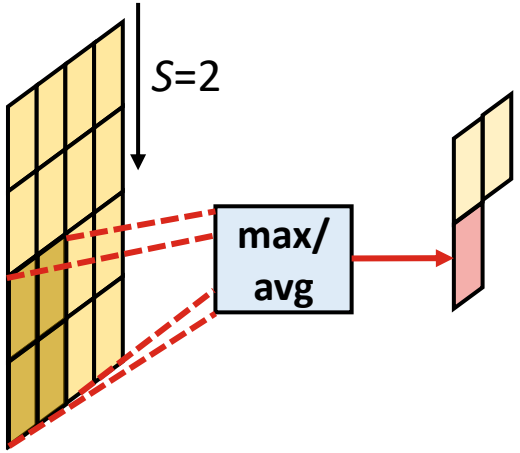
Max and average pooling

- *Max* and *average pooling* (*MaxPool* and *AvgPool*) are typically used for reducing spatial resolution (usually, stride $S > 1$)
  - MaxPool and AvgPool use sliding window like convolution, but convolution operation replaced by max or averaging operation

    MaxPool: $y_{i,j} = \max(\{x_{Si,Sj}, \ldots, x_{Si+K-1,Sj+K-1}\})$

    AvgPool: $y_{i,j} = \text{mean}(\{x_{Si,Sj}, \ldots, x_{Si+K-1,Sj+K-1}\})$

  - For multichannel input, performed separately for each input channel: $C_{OUT} = C_{IN}$



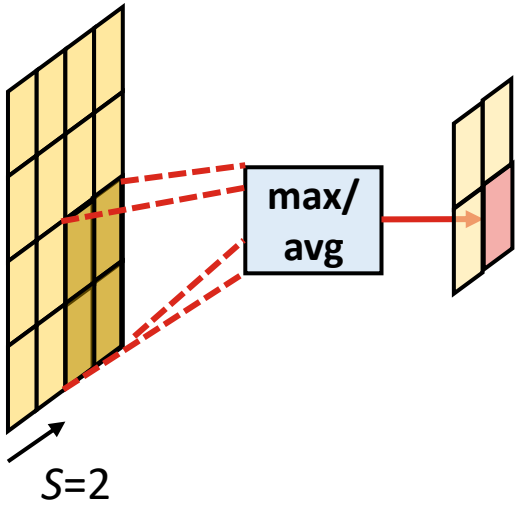
Max and average pooling

- *Max* and *average pooling* (*MaxPool* and *AvgPool*) are typically used for reducing spatial resolution (usually, stride $S > 1$)
  - MaxPool and AvgPool use sliding window like convolution, but convolution operation replaced by max or averaging operation

    MaxPool: $y_{i,j} = \max(\{x_{Si,Sj}, \dots, x_{Si+K-1,Sj+K-1}\})$

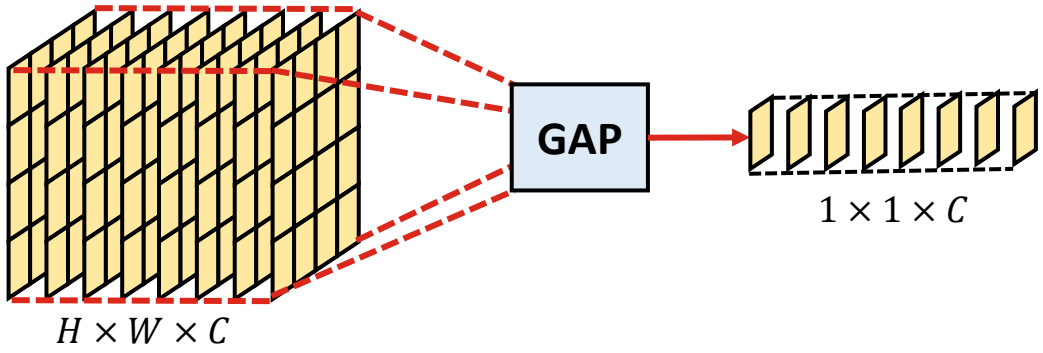
    AvgPool: $y_{i,j} = \text{mean}(\{x_{Si,Sj}, \dots, x_{Si+K-1,Sj+K-1}\})$

  - For multichannel input, performed separately for each input channel: $C_{OUT} = C_{IN}$



Global average pooling

- Global average pooling (GAP) performs average pooling over the full spatial dimensions
  - Same as average pooling where the averaging window height and width and the height and width of the input are the same
  - Used for reducing the multichannel input of size $H \times W \times C$ into a 1D vector of length C, usually to be used as input to a linear layer



Input data normalisation

- Neural networks perform best when the data has relatively narrow range of values: in typical CNNs, the input layer performs normalisation
  - Often 8-bit images of integer values $[0,255]$ are used as input: we could use *min-max normalisation* to scale the input to range $[0,1]$

$$x_i' = \frac{x_i - \min(x)}{\max(x) - \min(x)} = \frac{x}{255}$$

  - Another commonly used normalisation method is Z-score normalisation:

$$x_i' = \frac{x_i - \text{mean}(x)}{\sigma(x)}$$

  - For using Z-score normalisation, we need to first compute the mean and standard deviation σ from the training data

Batch normalisation

- Deep neural networks suffer from *vanishing gradient problem:* gradients tend to become very small, making the network unstable
  - In CNNs, training process is often stabilised using intermediate normalisation layers
  - Most common method is *batch normalization (BatchNorm* or *BN)*:

$$y_i = \gamma_i \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta_i$$

  - Mean μ_i, variance σ_i^2, and trainable parameters γ_i and β_i are computed for each channel i independently; ϵ is a small constant for better stability

Batch normalisation

- Deep neural n
  gradients tend
  - In CNNs, tra
    normalisati
  - Most comm

During training, mean μ_i and variance σ_i^2 are computed using that mini-batch of data; moving averages of μ_i and σ_i^2 are also computed to be used in the final trained network

$$y_i = \gamma_i \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta_i$$

- Mean μ_i, variance σ_i^2, and trainable parameters γ_i and β_i are computed for each channel i independently; ϵ is a small constant for better stability

Batch normalisation

- Deep neural networks suffer from *vanishing gradient problem:* gradients tend to become very small, making the network unstable
  - In CNNs, tra
  normalisati
  - Most comm

Trainable parameters γ_i and β_i are updated during training using the same optimisation as all the other trainable weights

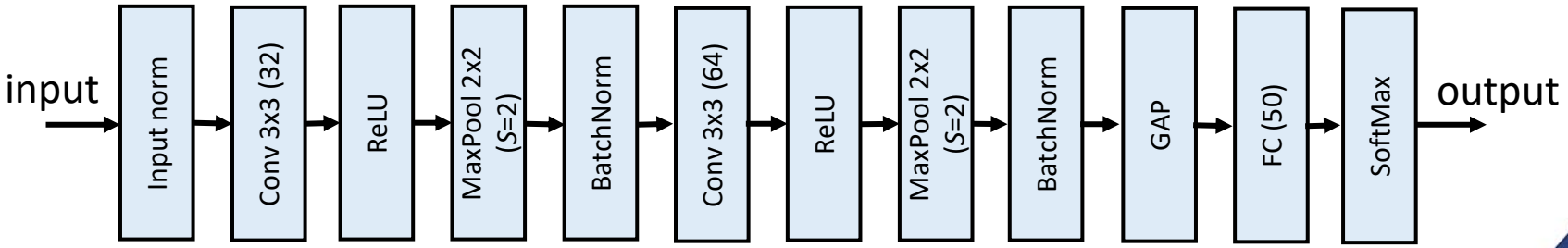
$$y_i = \gamma_i \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta_i$$

- Mean μ_i, variance σ_i^2, and trainable parameters γ_i and β_i are computed for each channel i independently; ϵ is a small constant for better stability

Break: any questions?

Simple example CNN

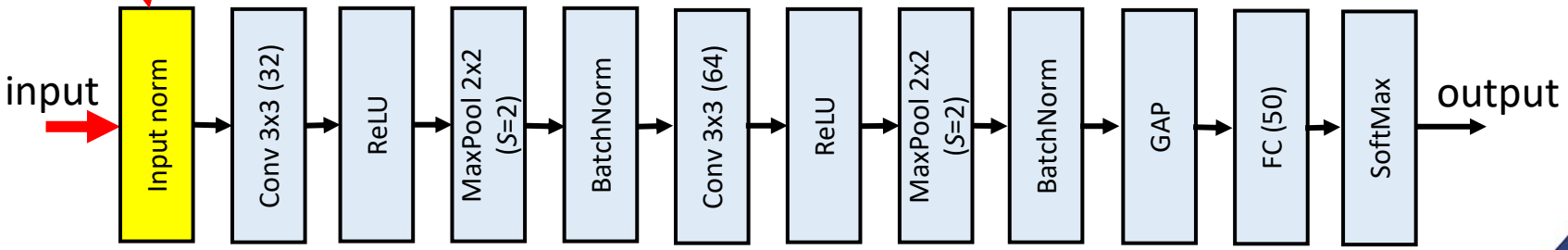
- Practical CNNs combine different types of layers
  - Spatial resolution usually decreasing towards the deeper layers, whereas channel dimension increasing
  - The last layers of image classification networks usually include a fully connected layer and SoftMax layer for classification



Simple example CNN

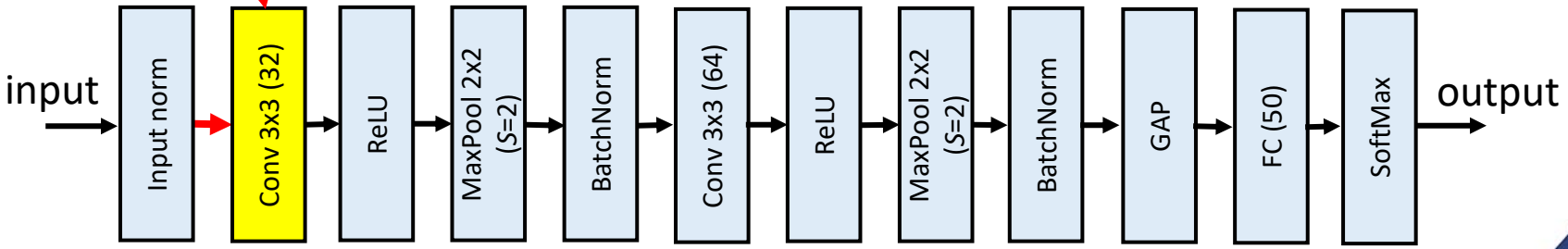
- Practical CNNs combine different types of layers
  - Spatial resolution usually decreasing towards the deeper layers, whereas channel dimension increasing
  - image classification networks usually include a fully connected layer and SoftMax layer for classification

Input: $H \times W \times 3$



Simple example CNN

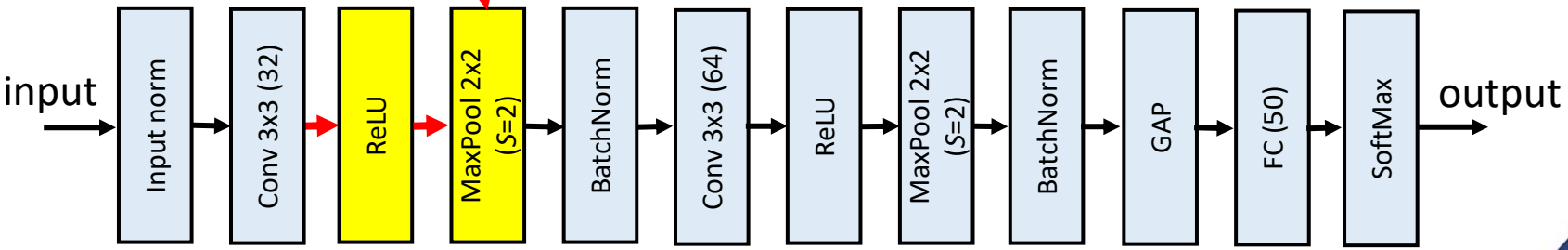
- Practical CNNs combine different types of layers
  - Spatial resolution usually decreasing towards the deeper layers, whereas channel dimension increasing
  - Output: $(H-2) \times (W-2) \times 32$
  - on networks usually include a fully connected layer and SoftMax layer for classification



Simple example CNN

- Practical CNNs combine different types of layers
  - Spatial resolution usually decreasing towards the deeper layers, w… …asing
  - T… …on networks usually include a fully connected layer and SoftMax layer for classification

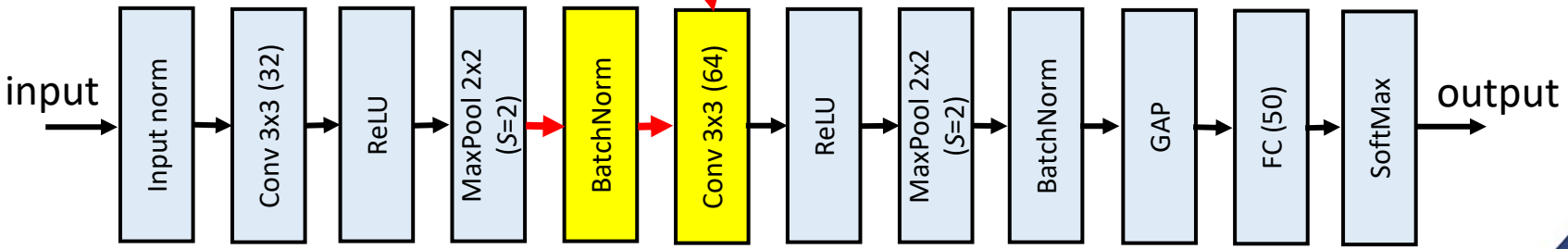
Output: $\frac{H-2}{2} \times \frac{W-2}{2} \times 32$



Simple example CNN

- Practical CNNs combine different types of layers
  - Spatial resolution usually decreasing towards the deeper layers, whereas cha
  - The last laye orks usually include a fully connected layer and SoftMax layer for classification

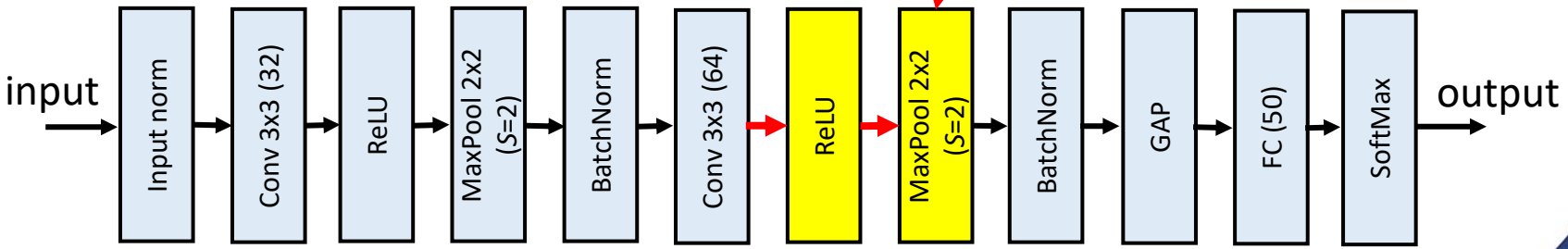
Output: $\frac{H-6}{2} \times \frac{W-6}{2} \times 64$



Simple example CNN

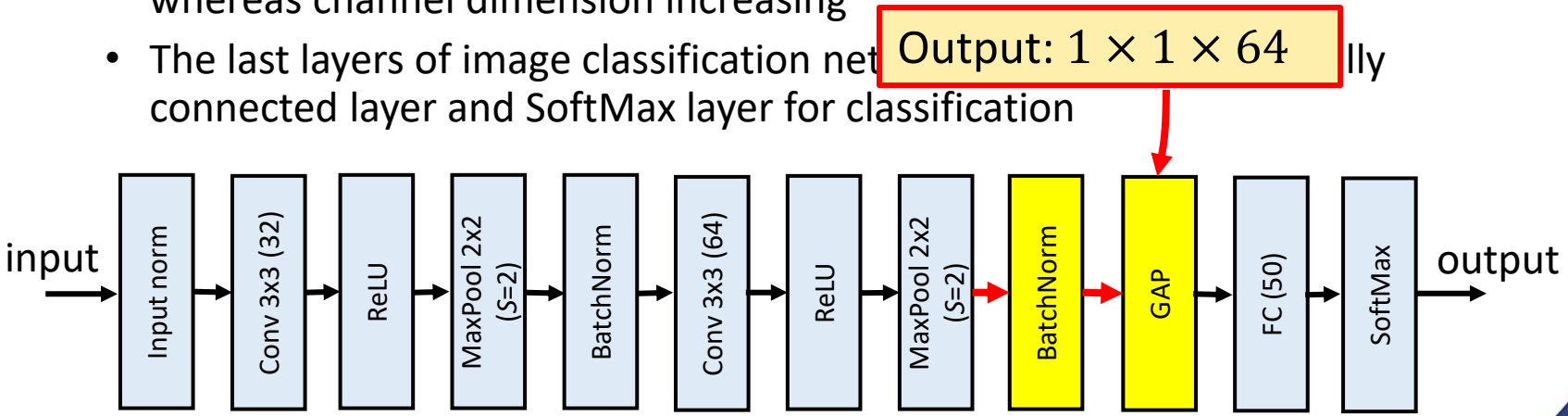
- Practical CNNs combine different types of layers
  - Spatial resolution usually decreasing towards the deeper layers, whereas channel dimen
  - The last layers of image include a fully connected layer and SoftMax layer for classification

Output: $\frac{H-6}{4} \times \frac{W-6}{4} \times 64$



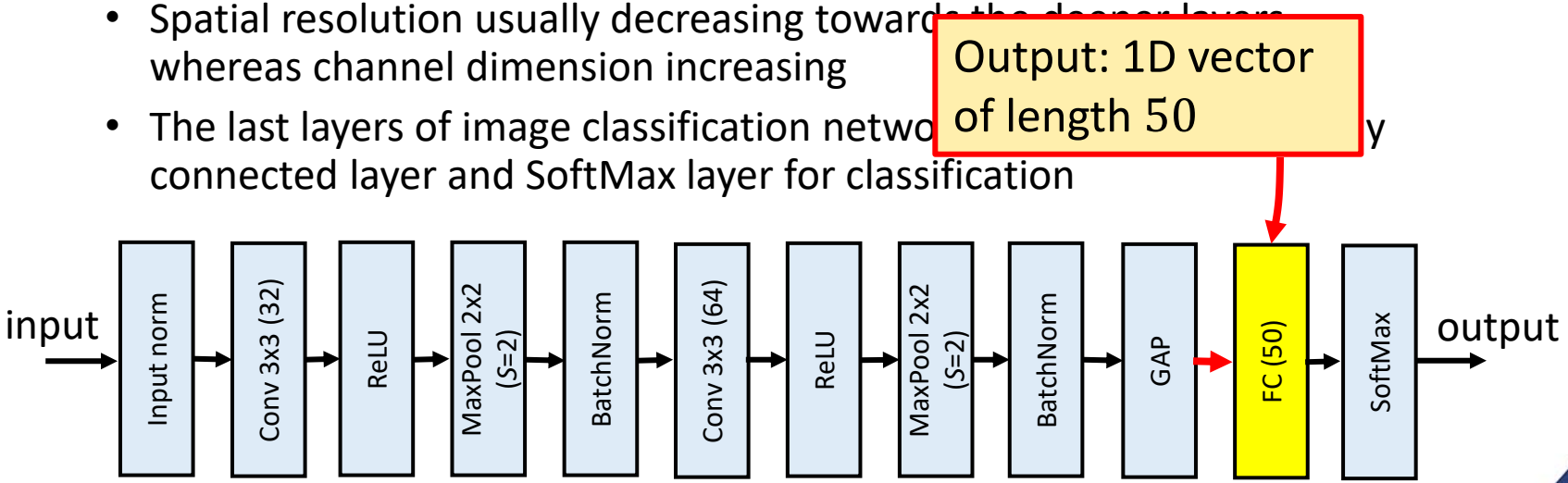
Simple example CNN

- Practical CNNs combine different types of layers
  - Spatial resolution usually decreasing towards the deeper layers, whereas channel dimension increasing
  - The last layers of image classification net lly connected layer and SoftMax layer for classification



Simple example CNN

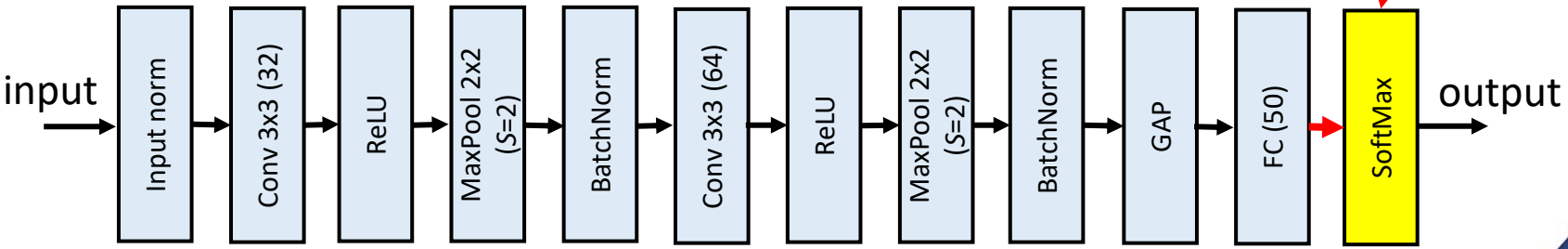
- Practical CNNs combine different types of layers
  - Spatial resolution usually decreasing toward the deeper layers whereas channel dimension increasing
  - The last layers of image classification netwo y connected layer and SoftMax layer for classification



Simple example CNN

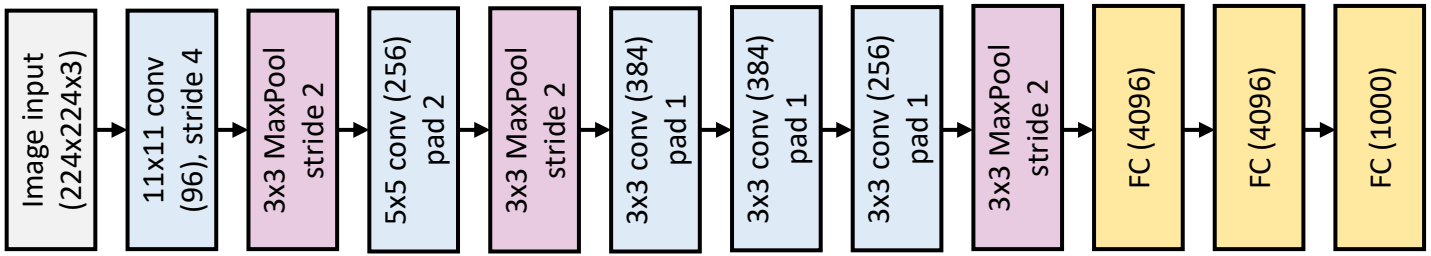
- Practical CNNs combine different types of layers
  - Spatial resolution usually decreasing to
    whereas channel dimension increasing
  - The last layers of image classification ne
    connected layer and SoftMax layer for classification

Output: probability distribution over 50 classes



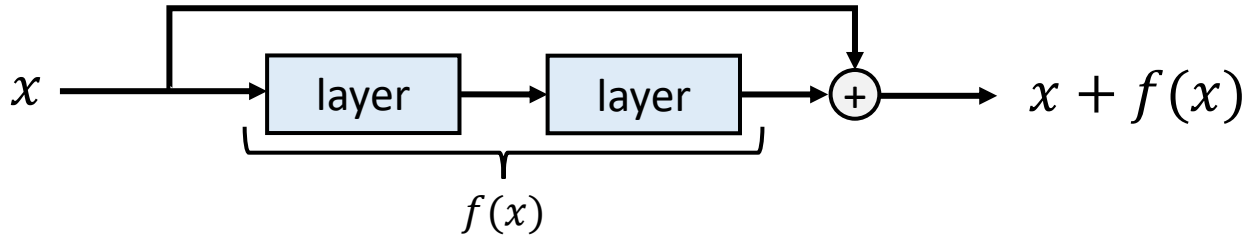
AlexNet

- Deep learning boom started when *AlexNet* by **Alex Krizhevsky** et al. was published in 2012
  - Significant improvement to the state-of-the-art in visual recognition tasks
  - AlexNet was not the first CNN, but it popularised CNNs for image classification tasks



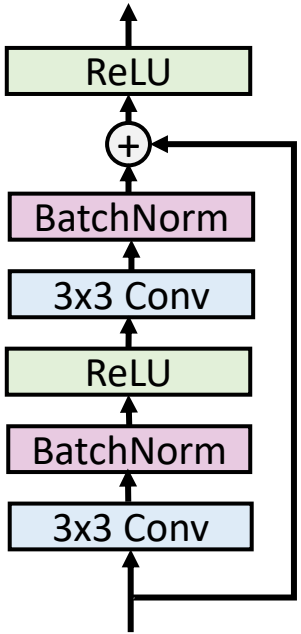
Residual neural network (ResNet)

- After AlexNet, improvement in accuracy was first attempted by simply adding more layers (e.g., *VGGNets* in 2014)
  - However, training of very deep networks turned out to be unstable
- Major improvement was achieved by **Kaiming He** et al. in 2015 using CNNs with *residual connections* to skip layers or group of layers
  - Output of the last layer in the main branch and the residual connection are connected using elementwise addition



Basic residual blocks

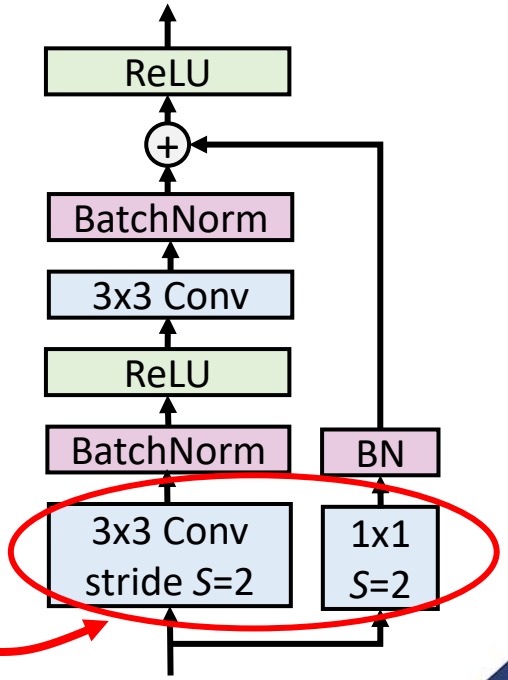
- The original ResNets built by combining basic residual blocks
  - Two 3x3 convolution layers, padding used to make input and output dimensions equal
  - ReLU for activation and BN for normalisation
- Basic blocks are also used for downsampling the spatial dimensions
  - 1x1 convolution layer with stride *S*=2 in the residual connection makes dimensions to match



Basic residual blocks

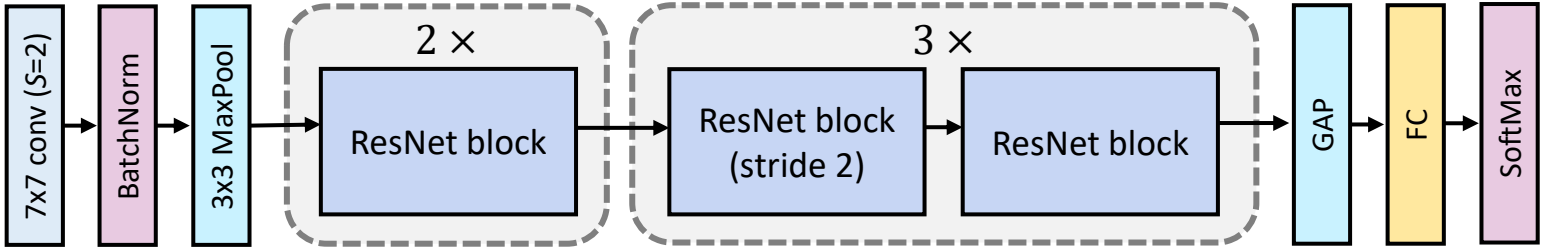
- The original ResNets built by combining basic residual blocks
  - Two 3x3 convolution layers, padding used to make input and output dimensions equal
  - ReLU for activation and BN for normalisation
- Basic blocks are also used for downsampling the spatial dimensions
  - 1x1 convolution layer with stride *S*=2 in the residual connection makes dimensions to match

Convolutions for downsampling



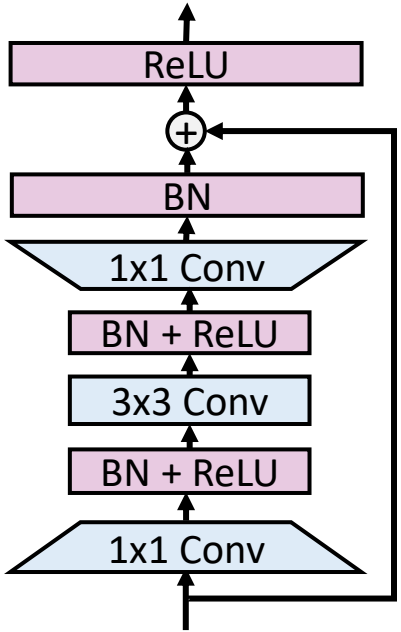
ResNet18

- Several models in the ResNet family have been proposed: ResNet18 is the smallest baseline model, built using the basic blocks
  - 8 ResNet basic blocks (3 for downsampling)
  - Counting the 3x3 convolution layers, the initial 7x7 convolution layer, and the final FC layer, there are 18 layers (BatchNorms, poolings, activations, and 1x1 downsampling convolutions excluded)



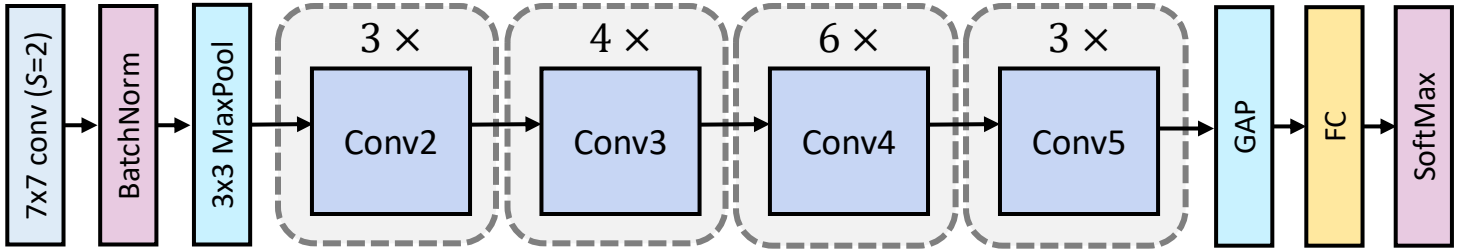
Bottleneck block

- The original ResNet study proposed also bottleneck blocks for deeper networks
  - 1x1 convolution used before 3x3 convolution to reduce the channel dimension
  - Another 1x1 convolution expands the channel dimension back
- Bottleneck blocks allow 3x3 convolution with smaller convolution kernel
  - Used in the deeper versions of ResNets



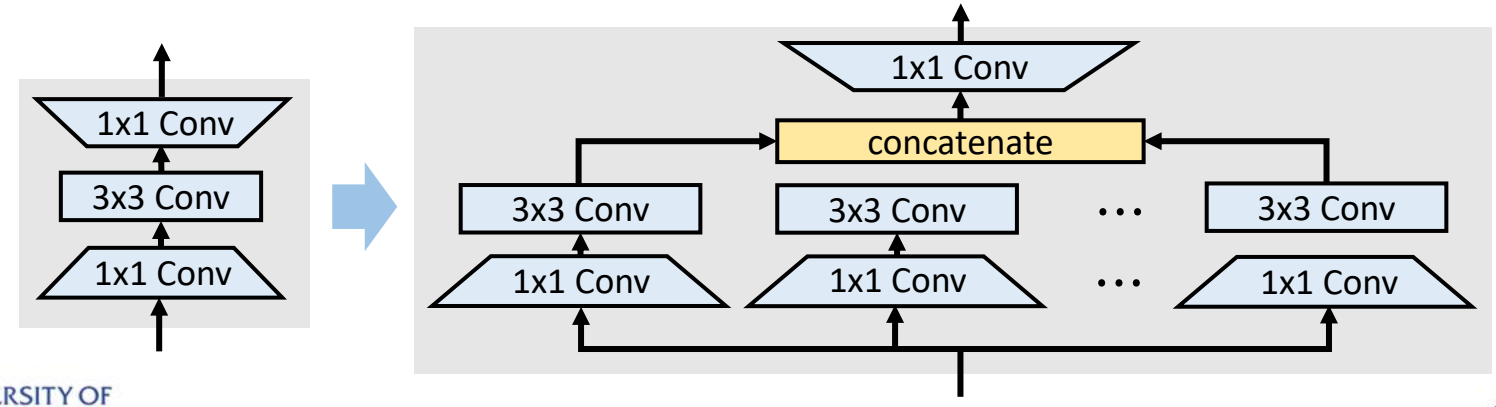
ResNet50 and above

- Deep ResNet models like ResNet50 and ResNet101 use bottleneck blocks instead of the basic ResNet blocks
  - Allows deeper structures without the number of weights to explode
  - ResNet blocks grouped as Conv2 to Conv5, where the groups have increasing channel dimension and decreasing spatial resolution
  - Channel upscaling and spatial downscaling in the first block of each group



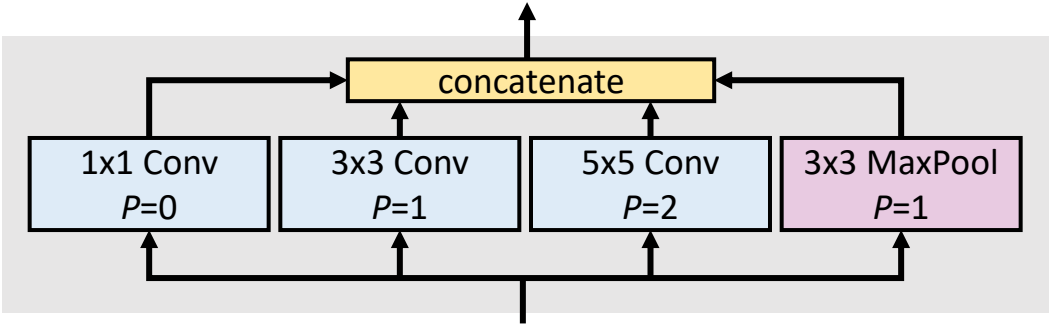
ResNeXt blocks

- *ResNeXt blocks* were developed to replace ResNet bottleneck blocks by performing several 3x3 convolutions in parallel instead of one
  - ResNeXt blocks have been found to improve performance of ResNets
  - The input and output dimensions are the same, so it is straightforward to replace ResNet bottleneck blocks with ResNeXt blocks



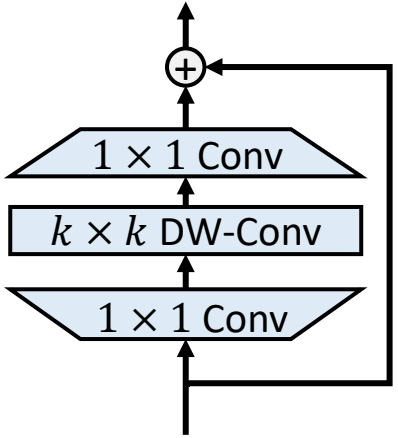
Inception networks

- *Inception modules* were developed based on the idea of using several different convolutions operations in parallel
  - *Inception networks* v1, v2, v3, and v4 using inception modules have been developed, as well as *InceptionResNet* using also skip connections
  - Inception networks have been used in several applications, but they have not become as popular as ResNets



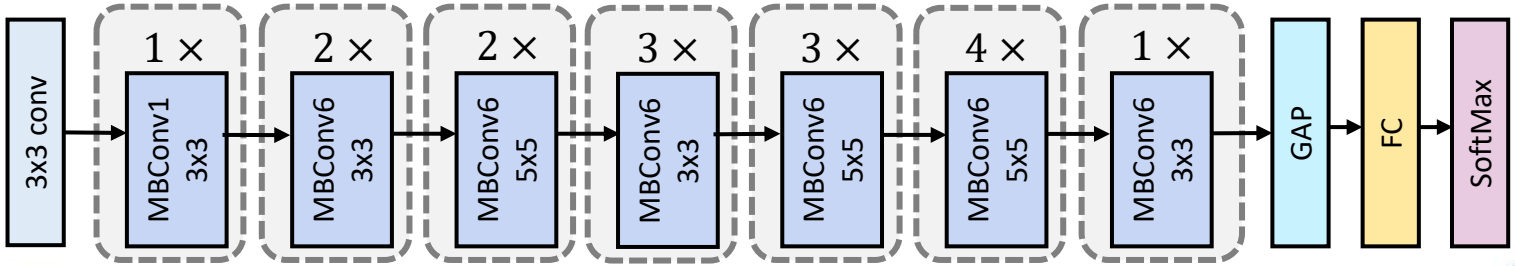
Mobile inverted bottleneck blocks

- *MobileNet* architecture introduced *mobile inverted bottleneck convolutional (MBConv)* blocks
  - First 1x1 convolution expands the channel dimension and the last 1x1 convolution shrinks it back
  - *Depthwise convolution (DW-Conv)* used for $k \times k$ convolution: each channel uses their own kernel
  - DW-Conv reduces the number of learnable weights radically: MBConv blocks works efficiently with less weights than the classic ResNet bottleneck blocks



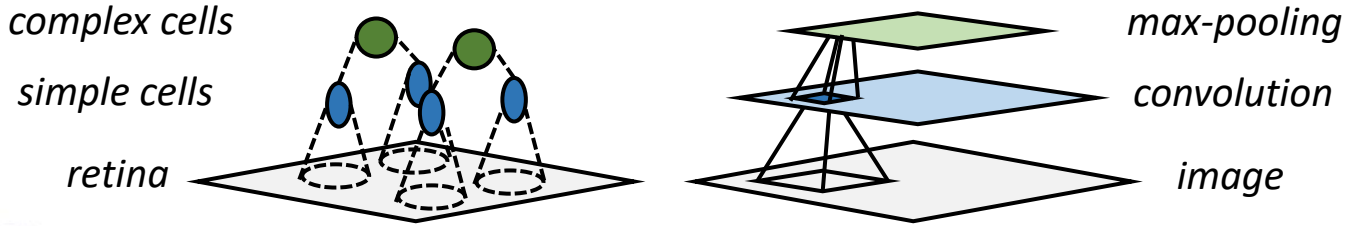
EfficientNet

- Although ResNets have been successful, their accuracy gains diminish above 100 convolution layers: how to scale networks efficiently?
  - **Mingxing Tan** et al. in 2019 studied scaling of CNNs in terms of depth (number of layers), width (number of channels) and image resolution
  - *EfficientNet* architecture using MBConv blocks as building blocks proposed: large EfficientNet models constructed from baseline EfficientNet-B0 through compound scaling of network dimensions



Biological inspiration of CNNs

- Neural research has suggested that human visual system processes information in a hierarchical manner resembling CNNs
  - Neurons in human eye perform filtering similar with convolution
  - First layers capture low level elements, like edges, whereas the deeper layers capture higher level elements with semantic meaning
- ResNets not claimed to be biologically inspired; however, recent research has found skip connections in biological neural networks



Summary

- Traditional fully connected perceptrons are not capable of handling very large inputs, such as images
  - Convolution operation processes only part of the input at once and uses a sliding window to cover the entire input
  - *Convolutional neural networks (CNN)* use convolution layers in combination with other types of layer for e.g., pooling and normalization
  - CNNs are popular especially in computer vision tasks, such as image classification
- The most prominent CNN building blocks and architectures outlined, with focus on *residual networks (ResNet)*

Questions and discussion